Week 5, Day 1: AWS Core (IAM, EC2, VPC, Security Groups) and Graph Basics

Prepared for Upendra Kumar Sharma | 4 hours, plus an optional 90-minute hands-on lab | Main topic: AWS foundations, IAM, EC2, VPC and security groups | DSA: Graph traversal, BFS and DFS

Today's Schedule

- **0:00 to 0:40** Study AWS foundations, the shared responsibility model and cost control (section 1).
- **0:40 to 1:20** Study IAM (section 2).
- **1:20 to 2:00** Study EC2 and storage (section 3).
- **2:00 to 2:50** Study VPC, subnets, routing and security groups (sections 4 and 5), and the target layout for your project (section 6).
- **2:50 to 3:50** DSA: graph templates and the 2 problems in section 8.
- **3:50 to 4:20** Answer the interview questions aloud (section 9) and add 10 flashcards.
- **Optional but strongly recommended (90 min):** the hands-on lab in section 7. Cloud knowledge you have only read about does not hold up in interviews.

Before anything else: protect your wallet. Create an AWS account only with a billing alert already planned. Turn on multi-factor authentication for the root user, create an IAM admin user (or use IAM Identity Center) for daily work, set a **budget with an email alert** (for example 5 or 10 dollars), and delete resources at the end of every session. Free tier terms change from time to time, so read the current terms on the AWS Free Tier page.

1. AWS Foundations

Global infrastructure

- **Region:** a geographic area (for example Mumbai, ap-south-1) made of several isolated data centers. Data stays in the region you choose unless you move it. Services, prices and latency differ by region, and compliance rules may restrict where data may live.
- **Availability Zone (AZ):** one or more data centers inside a region with independent power and networking, connected by fast links. **Spreading across at least two AZs** is the basic way to survive a data center failure.
- **Edge locations:** CloudFront (CDN) and Route 53 points of presence close to users.
- **Global vs regional services:** IAM, Route 53 and CloudFront are global. EC2, RDS, S3 buckets and most others live in a region.

Shared responsibility model

- **AWS is responsible for security of the cloud:** hardware, facilities, the virtualization layer, and managed service internals.
- **You are responsible for security in the cloud:** your data, IAM permissions, network rules (security groups), operating system patching on EC2, application security and encryption choices. For managed services such as RDS, AWS patches the engine, but you still manage access, backups settings and parameters.

How you interact with AWS: the console, the **CLI**, SDKs (for example the AWS SDK for Java), and **infrastructure as code** (CloudFormation, CDK, Terraform). Clicking is fine for learning, but real teams codify infrastructure so it is repeatable and reviewable.

Other basics

- **ARN (Amazon Resource Name):** the unique id of any resource, for example arn:aws:s3:::my-bucket. Policies refer to resources by ARN.
- **Tags:** key-value labels on resources, used for cost allocation and automation. Tag everything you create.
- **Pricing models: On-Demand** (pay per second or hour, no commitment), **Savings Plans and Reserved Instances** (commit to one or three years for a discount), **Spot** (spare capacity at a large discount, but can be reclaimed with short notice).
- **Well-Architected Framework pillars:** operational excellence, security, reliability, performance efficiency, cost optimization and sustainability. Quoting these when you justify a design shows maturity.
- **Cost tools:** Budgets (alerts), Cost Explorer, and the Pricing Calculator.

2. IAM (Identity and Access Management)

Building blocks

- **User:** a person or application with long-term credentials (a password, access keys). Prefer people to sign in through **IAM Identity Center** (single sign-on) with temporary credentials, and avoid long-lived access keys.
- **Group:** a set of users sharing permissions.
- **Role:** an identity with **no permanent credentials** that a trusted entity **assumes**, receiving **temporary credentials** from STS (Security Token Service). Used by EC2 instances, ECS tasks, Lambda functions, other AWS accounts and federated users.
- **Policy:** a JSON document that grants or denies permissions.

Policy structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::my-bucket/reports/*",
      "Condition": { "IpAddress": { "aws:SourceIp": "203.0.113.0/24" } }
    }
  ]
}
```

- **Effect** (Allow or Deny), **Action** (the API calls), **Resource** (which ARNs), and an optional **Condition** (for example by IP address, time, MFA presence or tag).

Types of policy

- **Identity-based policies** attach to a user, group or role and say what that identity may do.
- **Resource-based policies** attach to a resource (an S3 bucket policy, an SQS queue policy, a role **trust policy**) and say who may access it.
- **Permission boundaries** set the maximum permissions an identity can ever get.
- **Service control policies (SCPs)** in AWS Organizations set guardrails for whole accounts.
- A role has two policies: a **trust policy** (who may assume the role) and **permissions policies** (what the role may do).

How a request is decided (evaluation logic)

1. By default everything is **denied** (implicit deny).
2. An explicit **Allow** in an applicable policy grants access.
3. An explicit **Deny** anywhere **always wins** over any Allow.
4. Boundaries and SCPs must also allow the action, because they limit what an identity may be granted.

Good practice (interviewers ask for these)

- **Least privilege:** grant only the actions and resources needed. Start narrow and widen as needed, and use the IAM Access Analyzer and the policy simulator to check.
- **Never use the root user** for daily work. Lock it with MFA.
- **No access keys in code, images or Git.** An application running on AWS gets credentials from its **role**: an **instance profile** on EC2, a **task role** on ECS, an **execution role** on Lambda. The SDK finds them automatically.

- **Use MFA** for people, and rotate or delete unused keys.
- **Audit with CloudTrail**, which records API calls (who did what, when, from where).
- **Separate accounts** for production and development, managed through AWS Organizations.

Cross-account access: account B creates a role that trusts account A. A principal in account A assumes it and receives temporary credentials. No keys are shared.

3. EC2 and Block Storage

EC2 (Elastic Compute Cloud): virtual servers that you rent by the second.

- **Instance types:** named by family, generation and size, such as m6i.large. Families: **general purpose** (t and m), **compute optimized** (c), **memory optimized** (r), **storage optimized** (i, d), and **accelerated** (GPU). Names ending in g (for example t4g) use ARM-based Graviton processors, which are usually cheaper for the same performance, and Java runs on them well.
- **Burstable (t family):** a baseline CPU with credits for bursts. Good for small or spiky workloads, but a sustained heavy load can exhaust the credits and throttle.
- **AMI (Amazon Machine Image):** the template (operating system and software) that an instance boots from. You can build your own images with your application pre-installed.
- **User data:** a script that runs at first boot, for installing packages and starting services.
- **Access:** prefer **AWS Systems Manager Session Manager** over opening SSH to the internet. It needs an instance profile and no inbound ports. If you do use SSH, restrict the security group to your own IP.
- **Instance metadata service:** an endpoint inside the instance that returns its details and its role credentials. Require **IMDSv2** (session tokens), which protects against request forgery attacks that steal credentials.
- **Lifecycle: stop** (no compute charge, but EBS storage is still billed, and a public IP may change), **terminate** (deleted), and hibernate. Know the difference, since forgotten stopped volumes still cost money.

Storage for EC2

- **EBS (Elastic Block Store):** network-attached disks. **Persistent**, tied to **one AZ**, resizable, and backed up with **snapshots** (stored in S3). Types: gp3 (general SSD, the usual default), io2 (high, guaranteed IOPS), and HDD types for throughput.
- **Instance store:** disks physically attached to the host. Very fast, but **ephemeral** (data is lost when the instance stops or fails). Use for caches and temporary data only.
- **EFS** is shared network file storage across instances and AZs. **S3** is object storage (Day 2).

Scaling and availability

- **Auto Scaling Group (ASG):** keeps a desired number of instances running from a **launch template**, replaces unhealthy ones, and scales out and in on metrics such as CPU. Spread across several AZs.
- **Elastic Load Balancer (Day 3 covers it in depth):** an **Application Load Balancer (ALB)** distributes HTTP traffic across instances in several AZs and checks their health. An ASG behind an ALB is the classic highly available web tier.
- **Elastic IP:** a static public IPv4 address you can attach to an instance. Public IPv4 addresses now cost money, so avoid them unless needed.

Running a Spring Boot service on EC2: install Java or Docker, run the app as a systemd service or a container with a restart policy, send logs to CloudWatch with the agent, keep configuration in environment variables or Parameter Store, and give the instance a role instead of keys. Containers (ECS or Kubernetes) and serverless (Lambda) reduce this operational work, which Days 3 and 5 cover.

4. VPC (Virtual Private Cloud)

What it is: your own logically isolated network in AWS, where you define the address ranges, subnets, routes and firewall rules.

- **CIDR block:** the address range of the VPC, for example 10.0.0.0/16 (65,536 addresses). Subnets take slices of it, for example 10.0.1.0/24 (256 addresses). **AWS reserves 5 addresses in every subnet**, so a /24 has 251 usable. Plan ranges so that they do **not overlap** with other networks you may connect later.
- **Subnet:** a range within **one AZ**. A subnet is **public** if its route table sends internet traffic (0.0.0.0/0) to an **Internet Gateway**. Otherwise it is **private**.
- **Route table:** rules that decide where traffic goes. Every subnet is associated with one.
- **Internet Gateway (IGW):** lets resources with public IPs in public subnets talk to the internet in both directions.
- **NAT Gateway:** lives in a public subnet and lets resources in **private** subnets make **outbound** connections (updates, external APIs) without being reachable from the internet. It has an hourly cost plus a per-GB charge, so it is a common source of surprise bills.
- **VPC endpoints:** private connections to AWS services without going through the internet or NAT. **Gateway endpoints** (free) exist for S3 and DynamoDB. **Interface endpoints** (PrivateLink) cover many other services and cost money.
- **Connecting networks: VPC peering** (one-to-one), **Transit Gateway** (a hub for many VPCs), **Site-to-Site VPN** and **Direct Connect** for on-premises links.
- **VPC Flow Logs** record accepted and rejected traffic for troubleshooting and security.

- **Route 53** provides DNS, including private hosted zones inside a VPC.

The standard layout for a web application (three tiers, two or three AZs)

- **Public subnets:** the load balancer and NAT gateways.
- **Private application subnets:** EC2, ECS or EKS workloads.
- **Private data subnets:** RDS, ElastiCache and other data stores, with **no route to the internet**.

5. Security Groups and Network ACLs

Security group (the main firewall)

- Attached to a network interface (so effectively to an instance, a load balancer or a database).
- **Stateful:** if a request is allowed in, the response is automatically allowed out, and the other way round.
- **Allow rules only.** There is no deny rule. Anything not allowed is denied.
- Default: **all inbound denied, all outbound allowed**.
- A rule can name **another security group as its source**, instead of an IP range. This is the key technique: the database accepts connections only from members of the application security group, however many instances come and go.
- Changes take effect immediately.

Network ACL (NACL)

- Applies to a **subnet**, as a second, coarser layer.
- **Stateless:** you must allow return traffic explicitly (including ephemeral ports).
- Has **both allow and deny rules**, evaluated in numbered order.
- Most designs leave the NACL at its default and rely on security groups. NACLs are useful for blocking a specific bad IP range for a whole subnet.

Security group chaining for a three-tier app

- **ALB security group:** allow 80 and 443 from the internet.
- **Application security group:** allow the application port (for example 8080) **only from the ALB security group**.
- **Database security group:** allow 5432 **only from the application security group**.
- **Cache security group:** allow 6379 only from the application security group.
- **Result:** the database and Redis cannot be reached from the internet at all, and nothing else can talk to the application directly.

6. Target Layout for Your Project (Preview of Day 6)

- One **VPC** (10.0.0.0/16) with **two AZs**, each with a public subnet, a private application subnet and a private data subnet.
- **ALB** in the public subnets, in front of the gateway. Only the gateway is exposed.
- Application containers or instances in private subnets, with the **security group chain** above, and **IAM roles** for access to AWS services, with no keys.
- **RDS PostgreSQL** and **ElastiCache Redis** in the data subnets.
- **Cost reality check:** a NAT gateway, a managed Kafka cluster (Amazon MSK) and multi-AZ databases are expensive for a learning project. Options: run Kafka as a container on a small instance (for learning only), use SQS and SNS for a variant of the messaging, place instances in public subnets with tight security groups to avoid NAT costs, and **document what production would use**. State that trade-off openly in your README and in interviews.

7. Optional Hands-On Lab (90 Minutes)

1. **Secure the account.** Root MFA, an IAM admin user or Identity Center user, a budget alert. Then sign in as the new user.
2. **Set up the CLI.** Configure the AWS CLI for your user and run aws sts get-caller-identity to confirm who you are.
3. **Build a VPC.** Use the console's VPC wizard (the option that creates public and private subnets) with two AZs, **without a NAT gateway** to avoid costs. Open the route tables and find the route to the internet gateway for the public subnets. Write down how a private subnet differs.
4. **Create the security group chain** (ALB, application, database) with the rules in section 5.
5. **Launch an instance.** A small instance (for example t3.micro or t4g.small, within free tier if available) running Amazon Linux in a public subnet. Attach an **IAM role** with the AmazonSSMManagedInstanceCore policy and a read-only S3 policy. Connect with **Session Manager** (no SSH).
6. **Install Docker and run a container** (a simple web image, or one of your own services) on port 8080. Allow that port in the security group only from your own IP, and open it in a browser.
7. **Prove keyless access.** On the instance, run aws s3 ls. It works through the role, with no credentials stored anywhere. Then try an action your role does not allow, and read the access denied message.
8. **Write a least-privilege policy** for one bucket and one prefix, and test it in the IAM policy simulator.

9. **Clean up.** Terminate the instance, delete the security groups and the VPC, release any Elastic IP, and check the Billing page the next day.

Write down what you did and what surprised you. These notes become Day 6's deployment plan.

8. DSA: Graph Basics (2 Problems and the Templates)

Graph vocabulary and representations

- A graph has **vertices (nodes)** and **edges**, which are directed or undirected, and weighted or unweighted. Problems often hide a graph: a grid of cells, a set of dependencies, a network of users.
- **Adjacency list** (a list of neighbors per node): the default choice, with O(V + E) space. **Adjacency matrix:** O(V squared) space, quick edge lookup, good for dense graphs. **Edge list:** simple input format.
- A **grid** is an implicit graph: each cell has up to 4 neighbors (up, down, left, right), and you do not build the list.
- Traversal takes **O(V + E)** time. Always track **visited** nodes, so cycles do not cause infinite loops.

BFS (breadth-first search): a queue, level by level. Finds the **shortest path in an unweighted graph**. Mark a node visited **when you add it to the queue**, not when you remove it, or it can be queued several times.

```
void bfs(int start, List<List<Integer>> graph) {
    boolean[] seen = new boolean[graph.size()];
    Queue<Integer> queue = new ArrayDeque<>();
    queue.offer(start);
    seen[start] = true;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        for (int next : graph.get(node)) {
            if (!seen[next]) {
                seen[next] = true;
                queue.offer(next);
            }
        }
    }
}
```

DFS (depth-first search): recursion or a stack. Good for exploring everything reachable, connected components, cycle detection, and path problems. The recursive version can overflow the stack on very deep graphs, so know the iterative form.

```
void dfs(int node, List<List<Integer>> graph, boolean[] seen) {
    seen[node] = true;
    for (int next : graph.get(node)) {
        if (!seen[next]) dfs(next, graph, seen);
    }
}
```

When to use which: shortest number of steps in an unweighted graph means BFS. Connectivity, counting components, or exhaustive exploration means either one, and DFS is shorter to write.

Problem 1: Number of Islands

- **The problem:** a grid of land (1) and water (0). Count the groups of connected land cells (connected up, down, left or right).
- **Idea:** scan every cell. When you find unvisited land, **count one island** and **flood fill** it (DFS or BFS), marking every connected land cell as visited, so it is never counted again. Marking by overwriting the cell with 0 saves memory. Ask whether you may modify the input, or use a visited array.
- **Complexity:** O(R x C) time. Space is O(R x C) in the worst case for the recursion stack, so on very large grids use the BFS version or union find (Day 4).
- **Edge cases:** an empty grid, all water, all land, diagonal cells (not connected), a single row or column.

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == '1') {
                count++;
                sink(grid, r, c);
            }
        }
    }
    return count;
}

private void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] !=
'1') return;
    g[r][c] = '0';                      // mark visited
    sink(g, r + 1, c);
```

```
    sink(g, r - 1, c);
    sink(g, r, c + 1);
    sink(g, r, c - 1);
}
```

Iterative BFS flood fill (safe for large grids)

```
private void sinkBfs(char[][] g, int r0, int c0) {
    Queue<int[]> queue = new ArrayDeque<>();
    queue.offer(new int[]{r0, c0});
    g[r0][c0] = '0';
    int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        for (int[] d : dirs) {
            int r = cell[0] + d[0], c = cell[1] + d[1];
            if (r >= 0 && c >= 0 && r < g.length && c < g[0].length && g[r]
[c] == '1') {
                g[r][c] = '0';
                queue.offer(new int[]{r, c});
            }
        }
    }
}
```

Problem 2: Clone Graph

- **The problem:** make a deep copy of a connected undirected graph given one starting node. Each node has a value and a list of neighbors.
- **Idea:** use a **HashMap from original node to its copy**. When you meet a node, create its copy **and store it in the map before visiting its neighbors**. Then copy each neighbor (reusing the map entry if it exists). The map handles cycles and prevents infinite recursion.
- **Complexity:** O(V + E) time and space.
- **Edge cases:** a null input, a single node with no neighbors, a node connected to itself, cycles.

```
class Node {
    int val;
    List<Node> neighbors;
    Node(int val) { this.val = val; this.neighbors = new ArrayList<>(); }
}

private final Map<Node, Node> copies = new HashMap<>();

public Node cloneGraph(Node node) {
    if (node == null) return null;
    if (copies.containsKey(node)) return copies.get(node);

    Node copy = new Node(node.val);
    copies.put(node, copy);                      // register before recursing
    for (Node neighbor : node.neighbors) {
        copy.neighbors.add(cloneGraph(neighbor));
    }
    return copy;
}
```

Patterns to remember from today: a grid is a graph, count components with a loop plus a flood fill, and use a visited set (or a map from original to copy) to survive cycles.

9. Interview Questions to Answer Aloud

1. What are a Region and an Availability Zone, and how do you design for the failure of one AZ?
2. Explain the shared responsibility model with an example for EC2 and for RDS.
3. What is the difference between an IAM user, a group and a role? When do you use a role?
4. How does AWS evaluate a request? What happens when an Allow and a Deny both apply?
5. How can an application on EC2 read from S3 without storing access keys?
6. What are a trust policy and a permissions policy? How does cross-account access work?
7. EC2 pricing: On-Demand, Savings Plans or Reserved, and Spot. Which would you use for a steady service and which for a batch job?
8. EBS vs instance store: what happens to the data on stop and on failure?
9. What makes a subnet public or private? What does a NAT gateway do, and why can it become expensive?

10. Security group vs network ACL: stateful versus stateless, allow versus deny, and where each applies.
11. How would you make sure only the application tier can reach the database?
12. Design a VPC for a three-tier web application in two AZs. What goes in each subnet?
13. How would you reach a private instance without opening SSH to the internet?
14. How do you control costs when you run learning or test environments?

10. End-of-Day Checklist

- [ ] My AWS account has root MFA, an IAM user, and a billing alert.
- [ ] I can explain Regions, AZs, the shared responsibility model and the pricing models.
- [ ] I can explain IAM roles, policy structure, evaluation logic and why roles beat access keys.
- [ ] I can describe EC2 instance types, EBS vs instance store, Auto Scaling and the ALB.
- [ ] I can draw a VPC with public, private application and private data subnets, and explain the routing and security group chain.
- [ ] I solved Number of Islands and Clone Graph and can write the BFS and DFS templates from memory.
- [ ] (Optional) I completed the lab, proved keyless access, and cleaned up all resources.
- [ ] I wrote at least 10 flashcards from questions I got wrong.

Tomorrow (Week 5, Day 2): S3, RDS, ElastiCache, SQS and SNS, plus Course Schedule and topological sort.